

Zero Knowledge for Flock’s BLAKE3 Prover:

Construction, simulation, and concrete bounds

Alexander Lee

Zero-Knowledge Flock Implementation Candidate
July 2026

Abstract

The original Flock paper and implementation provide succinct proofs for batched BLAKE3 computation, but do not provide zero knowledge: the proof transcript contains deterministic functions of the witness [1]. This paper develops Flock-ZK for the same BLAKE3 circuit. BLAKE3 is the first target because it is Flock’s original benchmark and therefore gives a direct comparison between the ZK and non-ZK provers. A companion implementation provides a Rust prover, verifier, simulator, and extractor, together with Lean lemmas and replayable evidence.

The implementation registers two statements over the same batch of 256 single-block circuit instances. The batch statement preserves Flock’s benchmark shape but does not bind the trace to public hash outputs. The fixed-digest statement proves the relation

$$y_i = \text{BLAKE3}(x_i), \quad |x_i| = 64 \text{ bytes}, \quad |y_i| = 32 \text{ bytes},$$

for 256 public digests while hiding the messages. Honest and simulated proofs have the same format and are accepted by the same Flock-ZK verifier. There is no simulator-only verification rule; this is not the original non-ZK verifier.

The proof has two algebraic steps. First, a field-valued mask makes the PIOP messages independent of the witness after conditioning on the two mask values that are later opened. For the registered shape, the relevant determinant has degree at most 720, as derived in Section 4. Second, an affine translation of the PCS blinders preserves every opened value and every recursive codeword. These two translations reduce simulation to choosing the Fiat–Shamir challenges first, solving the dependent messages, and programming 18 fresh random-oracle points: one point for z , one for γ , and sixteen fold challenges. The simulator receives only the public statement.

In the classical programmable random-oracle model, the explicit algebraic and programming terms sum to less than $2^{-118.502}$ at a hash-query budget $Q_H = 2^{64}$, plus a separately stated oracle-boundary term. Thus the two registered profiles are computationally zero knowledge under the assumptions stated here. Knowledge security is separate: one L0 term caps the available standalone Fiat–Shamir reduction over $\mathbb{F}_{2^{128}}$ at 55.994 bits for the same query budget, before other knowledge-error terms are added. The stronger 100-bit deployment target is conjectural, not a theorem of this paper. At the registered batch size of 256, the current unoptimized ZK prover is 6.78 times slower than the current non-ZK prover on twelve threads, and its encoded proof is 9.21 times larger.

1 Result and scope

Result in one sentence. For the two parameter sets in Table 1, a proof generated from a valid witness is computationally indistinguishable from a proof generated from the public statement alone, in the classical programmable-random-oracle model.

Flock-ZK starts with BLAKE3 for scope, not because the masking method is incompatible with other hashes. A zero-knowledge theorem attaches to an exact relation, circuit layout, transcript, and PCS parameter set. BLAKE3 already had Flock’s most mature circuit and non-ZK benchmark, so it was the smallest target that permits both a complete implementation and a meaningful performance comparison.

The fixed-digest relation is

$$\mathcal{R}_{\text{fd}} = \{((y_1, \dots, y_{256}), (x_1, \dots, x_{256})) : y_i = \text{BLAKE3}(x_i), |x_i| = 64 \text{ bytes}, |y_i| = 32 \text{ bytes}\}.$$

The batch relation uses the same BLAKE3 circuit but has no externally fixed digest list or hash output. Its statement therefore exposes only the batch shape; the prover is free to choose the batch trace. The fixed-digest profile is the stronger application statement: it proves knowledge of preimages for public digests while hiding those preimages. In both cases the R1CS shape, packing convention, PCS parameters, and circuit digest are part of the registered statement family.

Throughout the paper, *computational zero knowledge* has its standard classical-ROM meaning: for every classical probabilistic polynomial-time distinguisher making at most Q_H oracle queries, the joint view of an honest proof and its oracle answers is computationally indistinguishable from the view produced by a simulator that knows only the statement. The simulator may program points that the distinguisher has not already queried. This is the IOP and Fiat–Shamir setting of Ben-Sasson, Chiesa, and Spooner [2]; the algebraic masking step is in the spirit of zero-knowledge sumcheck [3].

Table 1: The complete implementation scope of the ZK claim.

Family	Public input	Circuit / PCS	Status
BLAKE3 batch	batch size 256	$m = 22$, rate 1/2, batch log 6	computational ZK
BLAKE3 preimage	256 digests	$m = 22$, rate 1/2, batch log 6	computational ZK

The scope is deliberately narrow. The preimage circuit hashes one message of exactly 64 bytes and pins the BLAKE3 IV, counter, block length, and `CHUNK_START`, `CHUNK_END`, and `ROOT` flags. Unknown circuit digests, batch sizes, and PCS parameters are rejected before proving. SHA-256 statements, Keccak statements, longer BLAKE3 messages, hash chains, Merkle-path statements, recursion, side channels, and quantum random-oracle queries are not covered. This is an absence of a family-specific proof, not a claim that the construction cannot be adapted. Section 10 lists the obligations that must be re-established for each new family.

Construction at a glance. The argument follows the data seen by the verifier:

1. the BLAKE3 computation is written as an R1CS assignment z satisfying $(Az) \circ (Bz) = Cz$;
2. zerocheck reduces all multiplication constraints to evaluations at a transcript-derived point;
3. lincheck ties those evaluations back to the public matrices A, B, C and the same committed assignment z ;
4. ring switching converts the bit-level evaluation claims into packed extension-field claims, which Ligerito commits to and opens;
5. independent masks randomize every witness-dependent value while preserving the values that the verifier checks; and

6. the simulator samples those randomized values from the public statement and programs only fresh, domain-separated oracle points.

Sections 2–6 develop this construction in order. Section 7 measures its cost. Sections 8 and 9 treat extraction separately because witness privacy and knowledge soundness are different properties. Section 11 states exactly which obligations are proved in Lean, checked by executable certificates, or left as explicit assumptions.

Assumption 1 (Classical programmable random oracle). All injectively framed SHA-256 calls are modelled as one random function $\mathcal{H} : \{0,1\}^* \rightarrow \{0,1\}^{256}$. The adversary makes classical queries, and the simulator may program a point only before that point is queried. No QROM claim is made.

2 Relation and construction

Let z be the packed binary assignment for the block-diagonal BLAKE3 R1CS, embedded in $\mathbb{F}_{2^{128}}$, and write

$$a = Az, \quad b = Bz, \quad c_z = Cz.$$

The R1CS is satisfied when $a \circ b = c_z$. Zerocheck treats the vectors as multilinear polynomials and reduces the assertion $\widehat{a}(y)\widehat{b}(y) + \widehat{c}_z(y) = 0$ on the Boolean cube to terminal evaluations at a random point. Each round folds one or more variables and leaves a smaller claim. A false residual polynomial reaches the terminal identity only on a small exceptional set of challenges.

Zerocheck alone does not show that the prover used $a = Az$, $b = Bz$, and $c_z = Cz$. Lincheck supplies that link. It randomly combines the public matrix relations, asks for a partial evaluation of z , and reduces consistency to evaluation claims on the same committed assignment. Thus zerocheck checks the multiplicative constraints, while lincheck prevents the prover from inventing unrelated vectors that merely satisfy their product equation.

The resulting claims are opened with the ring-switched Ligerito PCS [4, 5]. The companion cheat sheet gives a high-level account of Ligerito, its relationship to Ligero, and ring switching.

For the fixed-digest family, the verifier derives an additional packed opening claim from the public digest list. Before the first challenge, the transcript binds that list, the padding rule, the witness layout, the circuit matrices, the parameter set, and a fresh 32-byte proof nonce.

The ZK prover adds four independent mask sources: the zerocheck mask P , the lincheck mask S , and the round-one masks S_c, S_h . The witness and mask commitments use independent hiding randomness, denoted μ and g . Every stream, including the proof nonce, is derived from a per-proof DRBG under a distinct label.

The main zerocheck mask is

$$\gamma P(X)Q^*(X), \quad Q^*(X) = q_0 + \sum_j q_j X_j.$$

Here P is a fresh witness-free polynomial, committed before the verifier samples γ . Its coefficients are uniform in $\mathbb{F}_{2^{128}}$ on a translated subcube with $d_{\log} = 12$ variables. The polynomial Q^* is fixed, public, affine, and nonzero on the Boolean cube. Multiplication by Q^* gives a degree-two mask with enough independent directions to randomize both coordinates of each degree-two zerocheck round message; Proposition 1 makes that statement quantitative. The challenge γ prevents the

prover from choosing P after seeing how it would need to cancel an invalid computation. At the terminal point ρ , the verifier checks

$$\widehat{a}(\rho)\widehat{b}(\rho) + \gamma P(\rho)Q^*(\rho),$$

and authenticates $P(\rho)$ against its earlier commitment. The mask therefore changes the visible proof distribution without becoming a free correction to the relation. The separate masks S_c, S_h cover the first zerocheck message, and S covers lincheck.

3 One random-oracle boundary

In Flock-ZK, Fiat–Shamir challenges, Merkle commitments, and proof of work are all implemented through the one random oracle of Assumption 1, but domain separation prevents them from using the same input. Each input begins with a role byte. A Merkle input also binds the wire-protocol identifier, proof nonce, commitment channel, recursion depth, node level, node index, payload length, and payload. A proof-of-work input is

$$\text{ROLE_POW} \parallel \text{state} \parallel \text{nonce}.$$

The proof nonce is absorbed immediately after the statement and before the first challenge.

This framing is part of the proof, not merely an encoding convention. It lets the extractor identify witness-leaf queries without relying on their order, prevents simulator programming from aliasing transcript or Merkle inputs, and makes proofs from different nonces or commitment channels use different oracle points. Native, recording, and programming backends implement the same frames. When no point is programmed they return identical bytes. Scalar and four-way SIMD implementations are differentially tested against the one-shot definition for every supported tail shape.

4 The PIOP translation

Fix the post-commitment challenge tuple χ . Let D be the coefficient space of P . If $n = m - 6$ is the number of masked zerocheck rounds, define

$$R_\chi : D \longrightarrow \mathbb{F}_{2^{128}}^{2n}$$

to be the map from mask coefficients to the $2n$ round-message coordinates. Let

$$L_\chi : D \longrightarrow \mathbb{F}_{2^{128}}^2$$

map the same coefficients to the two mask values revealed later: the initial masked claim and $P(\rho)$. The kernel $\ker L_\chi$ is simply the set of masks that change neither revealed value. Once $L_\chi(P)$ is fixed, these are the only mask changes still available. Their effect on the round messages is the subspace $R_\chi(\ker L_\chi)$.

Lemma 1 (Conditioned image dimension). *Let $N = \dim D$. If*

$$\text{rank} \begin{bmatrix} R_\chi \\ L_\chi \end{bmatrix} = 2n, \quad \text{rank } L_\chi = 2,$$

then

$$\dim R_\chi(\ker L_\chi) = 2n - 2.$$

Proof. The stacked-rank assumption gives $\dim(\ker R_\chi \cap \ker L_\chi) = N - 2n$, while $\text{rank } L_\chi = 2$ gives $\dim \ker L_\chi = N - 2$. Rank-nullity for R_χ restricted to $\ker L_\chi$ now gives

$$\dim R_\chi(\ker L_\chi) = (N - 2) - (N - 2n) = 2n - 2.$$

□

For the registered parameters, $m = 22$ and $n = 16$, so R_χ has 32 output coordinates. The sumcheck recurrence makes the two rows of L_χ dependent on the rows of R_χ , so the stacked map has rank at most 32. To prove that it has rank 32, it is enough to find a square 32×32 submatrix, called a *minor*, with nonzero determinant.

The symbolic evaluator builds the entries of this matrix as polynomials in the 32 verifier challenges. It follows the same straight-line recurrences as the concrete evaluator, while tracking either field values or conservative multivariate degrees. It contains no challenge-dependent inversion. The selected minor evaluates to a nonzero field element at one recorded challenge tuple, proving that its determinant is not the zero polynomial. Summing the per-row degree bounds gives total degree at most 720. Thus 720 is not a constraint count or a fitted constant; it is a conservative degree bound for this determinant.

Proposition 1 (Registered field-mask coverage). *For uniformly sampled $\chi \in \mathbb{F}_{2^{128}}^{32}$, the registered Flock-ZK shape satisfies the hypotheses of Lemma 1, except with probability*

$$\Pr_\chi[\text{coverage failure}] \leq \frac{720}{2^{128}} < 2^{-118.508}.$$

Consequently, after the two revealed mask values are fixed, the remaining mask freedom covers all $2n - 2 = 30$ independent round-message directions.

Proof. The nonzero minor has total degree at most 720. Schwartz–Zippel bounds the probability that it vanishes at a uniform challenge tuple by $720/|\mathbb{F}_{2^{128}}| = 720/2^{128}$ [6, 7]. Outside that zero set, the minor gives rank 32 for the stacked map, the separately checked leakage map has rank 2, and Lemma 1 applies. Section 11 records how the matrix, nonzero evaluation, and degree vector are replayed. □

Proposition 1 proves the PIOP masking step. It says nothing by itself about privacy at the PCS opening boundary; that is the next step.

5 The PCS translation

Write the initial combined opening vector as

$$F = (\mu \parallel z) + c g,$$

where $c \in \mathbb{F}_{2^{128}}$ is sampled after commitment. Let d be a witness difference that preserves every public opening claim. The aim is to replace z by $z + d$ without changing anything the verifier opens.

Lemma 2 (Opening-preserving translation). *Suppose $c \neq 0$ and the initial-level query system has full row rank. There is an affine bijection on the uniform PCS masks such that replacing z by $z + d$ leaves the queried initial rows and the entire combined vector F unchanged. Consequently every recursive codeword, root, authentication path, sumcheck value, and out-of-domain sample is unchanged.*

Proof. Set

$$\delta g_{\text{top}} = c^{-1}d.$$

Choose $\delta\mu$ to solve the queried initial-level encoding equations, and set

$$\delta g_{\text{lo}} = c^{-1}\delta\mu.$$

Because the field has characteristic two, $d + c\delta g_{\text{top}} = 0$ and $\delta\mu + c\delta g_{\text{lo}} = 0$. Thus $\delta F = 0$ globally. The defining equations for $\delta\mu$ also hold the separately opened initial rows fixed. Translation by these deltas is a bijection on uniform masks, so it preserves their distribution. $\square$

The registered verifier opens 218 initial-level positions. Each encoded lane starts with 512 independent mask symbols, so the query system is strictly underdetermined. There are 128 wide lanes, and the conservative rank audit leaves at least one independent element of $\mathbb{F}_{2^{128}}$ per lane in every relevant fresh leaf. Hence the minimum conditional entropy used at the Merkle boundary is $128 \cdot 128 = 16,384$ bits. No recursive entropy charge is needed because Lemma 2 makes the recursive codewords identical. The exceptional event $c = 0$ has probability 2^{-128} . Section 11 describes the executable query-rank and opening-field checks.

6 Simulator and concrete ZK bound

Once Proposition 1 and Lemma 2 are separated, the simulator is direct. It receives a public-statement capability containing the setup and digest list, but no witness type. It constructs an unrelated trace, overwrites the public-output region, samples the terminal zerocheck values, and solves the dependent messages in one pass. The patched trace is not an R1CS witness: running the honest prover on it is rejected. After the simulator programs the required fresh oracle points, however, the Flock-ZK verifier accepts the simulated proof.

For $m = 22$, zerocheck has $m - 6 = 16$ multilinear fold rounds. The simulator programs one oracle point for the univariate fold challenge z , one for the masking challenge γ , and one for each of the sixteen fold challenges ρ_j . This accounts for the bound of 18 programmed points.

A hybrid argument compares two distributions through intermediate thought experiments. Each row of Table 2 replaces one part of the honest proof by its simulated counterpart. The charge bounds the additional distinguishing advantage introduced by that replacement. Adding the charges bounds the distance between the first and last hybrids.

Table 2: Hybrid steps from an honest to a simulated transcript.

Hybrid	Replacement	Charge
G_0	honest proof	0
G_1	translate the PIOP mask	$720/2^{128}$
G_2	translate μ and g	$1/2^{128}$
G_3	cross the Merkle boundary	$Q_H/2^{16384} + \varepsilon_{\text{RO}}$
G_4	program fresh challenges	$18Q_H/2^{256}$
G_5	solve the terminal messages	$2/2^{128}$

The three algebraic numerators are 720, 1, and 2: the symbolic bad set, $c = 0$, and the two rejected values of the terminal challenge. Thus the part of the bound tracked explicitly by the executable ledger is

$$\varepsilon_{\text{explicit}}(Q_H) = \frac{723}{2^{128}} + \frac{18Q_H}{2^{256}} + \frac{Q_H}{2^{16384}}.$$

At $Q_H = 2^{64}$,

$$-\log_2 \varepsilon_{\text{explicit}}(2^{64}) = 118.502 \dots$$

The term ε_{RO} denotes collision and binding error at the framed 256-bit oracle boundary. It remains symbolic because a full count must include both adversarial queries and the protocol’s own Merkle, transcript, and grinding queries. It is not included in the quoted 118.502-bit figure, so that number is the explicit algebraic and programming ledger, not a complete standalone security level.

Theorem 1 (Scoped computational zero knowledge). *Assume the classical programmable random oracle of Assumption 1, the conditioned coverage of Proposition 1, the opening translation of Lemma 2, and freshness of the programmed points. Then both profiles in Table 1 are computationally zero knowledge. For every classical distinguisher making at most Q_H oracle queries,*

$$\text{Adv}_{\text{ZK}} \leq \varepsilon_{\text{explicit}}(Q_H) + \varepsilon_{\text{RO}}.$$

The simulator uses only the public statement, and the verifier is the same Flock-ZK verifier used for honest proofs. The original non-ZK Flock verifier is a different protocol path and is not meant to accept these proofs.

This theorem is in the classical programmable-ROM. It is not a QROM theorem, and it does not turn the ideal-oracle argument into a standard-model theorem for SHA-256.

7 Performance

Table 3 compares the registered Flock-ZK path with the current non-ZK Flock path at their common supported batch size. Both use the same 256 BLAKE3 compression inputs and the same repository revision. Measurements were taken on a 12-core Apple M4 Pro with 24 GB of memory, using twelve Rayon threads and the best of ten trials after one verified warm-up.

Table 3: Measured cost for a batch of 256 BLAKE3 compressions.

Metric	Non-ZK Flock	Flock-ZK	ZK / non-ZK
Prover time per batch	4.378 ms	29.674 ms	6.78×
Prover throughput	58,474/s	8,627/s	0.148×
Verifier time per batch	9.257 ms	5.306 ms	0.57×
Encoded proof size	265.7 KiB	2,447.9 KiB	9.21×

On one thread, the same harness measured 12.506 ms for non-ZK proving and 44.372 ms for Flock-ZK proving, a factor of 3.55×. Native scalar BLAKE3 took 58.5 ns per compression on this machine. A native batch of 256 therefore takes about 14.97 μ s, giving measured single-thread overheads of about 836× for current non-ZK Flock and 2,965× for the Flock-ZK path at this small batch.

The original Flock paper [1] reported less than 250× native overhead at the batch size that maximized throughput. It benchmarked powers of two through $2^{18} = 262,144$ BLAKE3 compressions and reached its peak single-thread throughput at 2^{16} ; the construction did not state a hard maximum batch size. Those figures use much larger batches, different hardware, and an earlier implementation, so they are context rather than a direct baseline for Table 3. In particular, multiplying 250 by 6.78 would not be a measured Flock-ZK overhead.

The Flock-ZK path is deliberately oriented toward auditability and still uses naive round-pair kernels. The table measures the implementation candidate as it exists; it does not claim that the observed factor is an intrinsic cost of zero knowledge. The implementation companion documents the matched benchmark procedure.

8 Extraction and simulation extractability

Zero knowledge means that a proof reveals nothing about the witness beyond the truth of the statement. Knowledge soundness asks the complementary question: can a successful prover be used to recover a witness? The two properties use different arguments and have different concrete bounds.

The recording oracle observes the framed leaf queries used to construct the witness commitment. Given a complete valid codeword, the extractor rebuilds the interleaved word, applies the inverse NTT lane by lane, re-encodes the candidate, checks the configured unique-radius condition, and returns the witness half. The system-specific step then reads the 512 message bits of each instance and independently verifies

$$\text{BLAKE3}(x_i) = y_i.$$

Honest recorded commitments yield the true preimages; simulated commitments do not extract.

The executable decoder is an exact-word decoder and candidate validator. It is not a complete Reed–Solomon decoder for every noisy word inside the theoretical radius. The general noisy-word step therefore comes from the Ligerito proximity and unique-decoding argument, not from the current decoder routine.

For simulation extractability, the simulator records

$$(\textit{statement digest}, \textit{proof nonce}, \textit{protocol identifier})$$

for each simulated proof. Extraction refuses every recorded prefix and is attempted only on a fresh one. The implemented property is therefore *fresh-prefix weak simulation extractability*. Same-prefix simulation extractability is not claimed.

9 Knowledge-security limit

The knowledge ledger contains an interactive L0 fold term

$$\kappa_\Gamma = \frac{257}{2^{128}},$$

or 119.994 bits. The classical Fiat–Shamir reduction multiplies this term by $Q_H + 1$. At $Q_H = 2^{64}$ this term alone becomes

$$-\log_2((Q_H + 1)\kappa_\Gamma) = 55.994 \dots \text{ bits}.$$

Thus 55.994 bits is a ceiling imposed by one term, not a completed lower guarantee for the whole argument. Every additional positive knowledge-error term can only reduce the final number of provable bits.

The 64-bit loss already occurs in the classical reduction. It comes from guessing which of the adversary’s Q_H oracle queries carries the challenge used in the successful proof, which contributes the factor $Q_H + 1$. No quantum adversary is needed for this loss. Thus “classical loss” means that

the stated standalone knowledge theorem is already only 55.994 bits against classical attackers. It does not mean that the protocol is post-quantum secure. The paper gives no quantum-random-oracle reduction and therefore makes no knowledge-security claim against quantum adversaries.

The standalone implementation profile must be labelled *100-bit conjectured classical knowledge security*. Its reduction ceiling is 55.994 bits before the remaining terms are added. A protocol with an unbiased post-commitment beacon could avoid this particular Fiat–Shamir query loss, and a wider challenge field could increase the margin. Neither change is implemented or claimed here. Grinding, a small- Q_H convention, generic parallel repetition, and deleting one fold challenge do not repair the standalone bound.

10 Extending the ZK construction

The masking code is reusable, but the ZK claim does not transfer automatically to another hash, circuit, batch size, or PCS shape. A new family such as SHA-256 or Keccak requires all of the following steps:

1. define the exact relation, public inputs, padding, and fixed hash parameters;
2. bind the circuit digest, witness layout, PCS parameters, and complete transcript schema;
3. regenerate the symbolic PIOP map and prove a nonzero determinant with an explicit all-challenge degree bound;
4. prove that the PCS mask translation is solvable for every query set the Flock-ZK verifier can derive, and recompute the unopened-leaf entropy;
5. construct a simulator whose interface accepts only the public statement and whose output is accepted by the unchanged family verifier;
6. enumerate every verifier-visible field and prove the hybrid composition, including random-oracle programming and grinding order;
7. implement the relation-specific extractor and recompute the complete ZK and knowledge-error ledgers; and
8. add a fail-closed certificate entry only after the family-specific negative controls and end-to-end certification pass.

The existing SHA-256 and Keccak encoders do not satisfy these obligations merely by existing. The algebraic masking strategy is expected to be reusable, but those statement families remain outside the ZK claim until their own certificates, simulators, extractors, and error ledgers are established.

11 Implementation evidence and limitations

The paper is accompanied by a Flock-ZK implementation and evidence suite. The repository documentation records exact wire identifiers, circuit digests, schema numbers, artifact paths, build commands, and benchmark commands. Those details are kept out of the mathematical argument because they change with implementation maintenance, while the claims below depend on their audited contents.

The evidence is deliberately layered. Lean proves reusable mathematical statements. Rust constructs and replays the matrices and translations for the registered parameters. The paper supplies the protocol-specific composition and states the random-oracle assumptions. Table 4 separates these roles.

Table 4: Formalization and executable-evidence status.

Claim	Machine support	Status
Uniform affine masking and bad-set composition	Generic Lean theorems	Proved in Lean; no protocol layout is encoded there
Schwartz–Zippel degree budget	Generic Lean theorem; Rust supplies the determinant data	Generic implication proved in Lean
Registered PIOP coverage	Symbolic Rust matrix, nonzero minor, and replayed degree vector	Protocol-specific certificate, not an end-to-end Lean theorem
PCS opening translation	Paper lemma and exact Rust translator tests	Registered fixture checked; general verifier-derived query proof remains open
Full ZK composition	Simulator tests, hybrid ledger, and paper argument	Classical-ROM theorem under the stated assumptions; not one Lean theorem
Extraction	Recording oracle and executable candidate decoder	Exact-word path implemented; general noisy decoding remains theoretical

Thus Theorem 1 is not presented as a single end-to-end Lean proof. Lean checks its generic masking, bad-set, and degree-budget ingredients. The exact Flock-ZK instantiation also relies on replayable Rust certificates, the paper proofs above, and the classical programmable-random-oracle model.

The executable evidence ties the numerical claims to the implementation, but does not replace cryptographic review. The remaining limits are:

- The ZK theorem is classical-ROM only; there is no QROM analysis.
- The standalone knowledge reduction over $\mathbb{F}_{2^{128}}$ is capped at 55.994 bits at $Q_H = 2^{64}$ before other terms are added. This is not a completed 100-bit theorem.
- The executable decoder validates candidates but does not implement general arbitrary-error Reed–Solomon decoding.
- The exact PCS translator is exercised on a fixture; a standalone proof for every query set derived by the Flock-ZK verifier remains a review obligation.
- Only the two exact registry entries are enabled. Other Flock statement families remain non-ZK.
- Side-channel resistance, recursive composition, and same-prefix simulation extractability are out of scope.

Within these limits, the conclusion is precise: for the registered BLAKE3 batch and fixed-digest relations, Flock now has a public-statement-only simulator whose proofs are accepted by the Flock-ZK verifier, together with an explicit distribution argument for every witness-dependent part of the transcript. That is the sense in which these Flock profiles are zero knowledge.

References

- [1] Succinct Labs. *Flock*. Historical paper at repository commit 73f7202. <https://github.com/succinctlabs/flock/blob/73f7202/paper/flock-paper.pdf>.
- [2] E. Ben-Sasson, A. Chiesa, and N. Spooner. *Interactive Oracle Proofs*. TCC 2016; IACR ePrint 2016/116.
- [3] A. Chiesa, M. A. Forbes, and N. Spooner. *A Zero Knowledge Sumcheck and its Applications*. IACR ePrint 2017/305.
- [4] A. Novakovic and G. Angeris. *Ligerito: A Small and Concretely Fast Polynomial Commitment Scheme*. IACR ePrint 2025/1187.
- [5] B. E. Diamond and J. Posen. *Polylogarithmic Proofs for Multilinears over Binary Towers*. IACR ePrint 2024/504.
- [6] J. T. Schwartz. *Fast Probabilistic Algorithms for Verification of Polynomial Identities*. Journal of the ACM 27(4), 701–717, 1980.
- [7] R. Zippel. *Probabilistic Algorithms for Sparse Polynomials*. EUROSAM 1979, LNCS 72, 216–226.